

Coding Problem 1: The "EcoSmart" Home Hub Controller

Scenario Background:

You are the lead architect for *EcoSmart*, a company that manufactures a central hub to manage various smart home devices. Your hub needs to control lights, security cameras, and smart locks.

Some of these devices are wired directly into the house's electricity, while others rely on internal batteries. Every night at 2:00 AM, the central hub runs a "System Diagnostics Routine." During this routine, the hub must trigger every device to perform its primary action to ensure it is working, and it must also check the battery levels of any device that relies on battery power, triggering a recharge alert if the battery is dangerously low.

Implementation Constraints & Requirements:

1. Inheritance Structure (The "Is-A" Relationship):

- Create an abstract base class called `SmartDevice`. It must contain:
 - A protected String `deviceId`.
 - A protected String `deviceName`.
 - A constructor to initialize these fields.
 - An abstract method `public abstract void runDiagnostic()`.
- Create three concrete classes that extends `SmartDevice`:
 - `SmartLight`
 - `SmartCamera`
 - `SmartLock`

2. Interface Definition (The "Can-Do" Behavior):

- Create an interface called `BatteryOperated`. It must contain:
 - `int getBatteryLevel()`
 - `void triggerRechargeAlert()`
- The `SmartLight` is hardwired into the ceiling. It does **not** use batteries.
- The `SmartCamera` and `SmartLock` are wireless. You must explicitly make these two classes implements `BatteryOperated`. (*Assume you can pass a battery level into their constructors for testing purposes*).

3. Polymorphism & The Hub Manager:

- Create a class called HomeHub.
 - Write a method with the exact signature: `public void executeNightlyRoutine(SmartDevice[] devices)`.
 - **The Polymorphism Constraint:** Inside this method, you must iterate through the array. For every device, you must call its polymorphic `runDiagnostic()` method.
 - However, if the device in the loop happens to be `BatteryOperated`, you must also check its battery level. If the battery is strictly less than **20%**, you must call `triggerRechargeAlert()`. You must achieve this using safe type-checking (`instanceof`) and downcasting.
-

Why this makes a great interview/study question:

1. **Separation of Concerns ("Is-A" vs. "Can-Do"):** It forces the developer to understand *why* we use both abstract classes and interfaces. A `SmartCamera` *is a* `SmartDevice`, so it inherits the ID and Name. But a `SmartCamera` *can be* `BatteryOperated`. Because Java does not support multiple inheritance for classes, interfaces are the only way to share this "battery" behavior with `SmartLock` without forcing it onto `SmartLight`.
2. **Dynamic Method Dispatch:** The `executeNightlyRoutine` method proves the power of Polymorphism. The `HomeHub` doesn't need a massive switch statement checking if (`device == SmartLight`). It just calls `device.runDiagnostic()`, trusting the JVM to execute the correct child method.
3. **Safe Downcasting (`instanceof`):** It creates a scenario where standard class polymorphism isn't enough. Because `SmartDevice` doesn't have a `getBatteryLevel()` method, calling it directly on the device variable would cause a compilation error. The developer must prove they know how to safely check capabilities at runtime using `instanceof` and cast the object to the interface type to unlock those specific methods.

Coding Problem 2: The "AeroLogix" Drone Fleet (Interface Segregation & Default Methods)

Scenario Background:

You are architecting the navigation software for *AeroLogix*, an automated package delivery network. The fleet consists of standard Quadcopters, wheeled City Rovers, and experimental Hybrid VTOL (Vertical Take-Off and Landing) craft.

All drones share basic identification and cargo tracking, but their traversal logic differs drastically. The aviation authority requires all flying drones to explicitly request airspace clearance before taking off, a logic that is identical across all flying models.

Implementation Constraints & Requirements:

1. Base Structure:

- Create an abstract class `DeliveryDrone` with a `String droneld` and an abstract method `public abstract void deliverPackage()`.

2. Interface Segregation:

- Create an interface `Airborne`. It must contain an abstract method `void flyToDestination()`.
- *Constraint:* You must utilize Java 8+ features to include a default method in `Airborne` called `requestAirTrafficClearance()` that prints a standard clearance message.
- Create an interface `GroundBased`. It must contain an abstract method `void navigateSidewalks()`.

3. Concrete Implementations:

- Create `Quadcopter` which extends `DeliveryDrone` and implements `Airborne`.
- Create `CityRover` which extends `DeliveryDrone` and implements `GroundBased`.
- Create `HybridVTOL` which extends `DeliveryDrone` and implements **both** `Airborne` and `GroundBased`.

Why this makes a great interview/study question:

- **The "Diamond Problem" Resolution:** It forces developers to understand that while Java prevents multiple inheritance of *state* (classes), it actively encourages multiple inheritance of *behavior* (interfaces). The `HybridVTOL` cleanly represents a system with overlapping capabilities.
- **Default Methods:** It highlights a critical modern Java feature. Before Java 8, adding `requestAirTrafficClearance()` to an interface would break all existing implementations. Default methods allow for backward-compatible interface evolution and shared logic without abstract classes.

Coding Problem 3: The "VaultGuard" State Machine (Encapsulation & Enums)

Scenario Background:

You are building the firmware for a bank vault door. The door operates strictly through a State Machine. A door can be OPEN, CLOSED, or LOCKED.

Engineering quality is paramount. A critical security flaw exists if a developer can bypass the physical mechanics (e.g., transitioning a door directly from OPEN to LOCKED without first closing it). You must design a class that entirely encapsulates the door's state and uses custom exceptions to reject illegal operations.

Implementation Constraints & Requirements:

1. State Representation:

- Create an enum called `DoorState` with values OPEN, CLOSED, and LOCKED.

2. Custom Exception:

- Create an unchecked exception called `IllegalStateTransitionException`.

3. Strict Encapsulation:

- Create a `VaultDoor` class. The internal `DoorState` must be private.
- Provide exactly three state-mutating methods: `closeDoor()`, `lockDoor()`, and `unlockDoor()`.
- *Constraint:* You cannot provide a "setter" for the state.
- *Constraint:* Inside `lockDoor()`, you must explicitly check if the door is CLOSED. If it is OPEN, throw the `IllegalStateTransitionException` with a descriptive message.

Why this makes a great interview/study question:

- **True Encapsulation:** Junior developers often auto-generate getters and setters for every private field, completely defeating the purpose of encapsulation. This problem forces them to realize that state should only mutate via validated, domain-specific behavior methods (`lockDoor()`), not data-assignment methods (`setState()`).
 - **Fail-Fast Engineering:** Throwing a custom `RuntimeException` immediately upon an illegal request ensures the system fails fast and loudly, preventing silent data corruption in a critical enterprise system.
-

Coding Problem 4: The "FinTech" Routing Engine (Strategy Pattern & Composition)

Scenario Background:

You are writing a payment processing engine for an e-commerce backend. The system needs to process transactions via Credit Card, PayPal, and Cryptocurrency.

Instead of writing a massive, hard-to-maintain switch statement inside your checkout logic, you must use the **Strategy Pattern**. This relies on the OOP principle of "Favor Composition over Inheritance," allowing the payment algorithm to be swapped dynamically at runtime without altering the core processor class.

Implementation Constraints & Requirements:

1. The Strategy Interface:

- Create an interface `PaymentStrategy` with a method `boolean processPayment(double amount)`.

2. Concrete Strategies:

- Create `CreditCardStrategy` and `CryptoStrategy` that implement this interface. (Return `true` and print a specific processing message).

3. The Context Class (Composition):

- Create a `TransactionProcessor` class.
- *Constraint:* It must maintain a private `PaymentStrategy` strategy variable.
- Provide a constructor to inject the strategy, and a setter `setPaymentStrategy()` to allow swapping the strategy at runtime.
- Create a method `executeTransaction(double amount)` that delegates the work to the currently loaded strategy.

Why this makes a great interview/study question:

- **Open/Closed Principle (SOLID):** If the company wants to add "Apple Pay" tomorrow, the developer does not need to touch the `TransactionProcessor` at all. They simply create an `ApplePayStrategy`. The core system is closed for modification but open for extension.
- **Composition vs. Inheritance:** It directly demonstrates why composition is superior for changing behaviors. You cannot change a class's parent class at runtime, but you can easily swap out an injected object.

Coding Problem 5: The "DataStream" ETL Pipeline (Bounded Generics)

Scenario Background:

You are building an Extract, Transform, Load (ETL) pipeline. The pipeline ingests abstract `DataPayload` objects. However, specific parsers are designed strictly for JSON or XML data.

To prevent `ClassCastException` errors at runtime, you must utilize Java Generics. You need to enforce *Compile-Time Type Safety* so that an XML payload cannot be accidentally passed into a JSON processing unit.

Implementation Constraints & Requirements:

1. The Domain Models:

- Create an abstract class `DataPayload` with a method `String getRawContent()`.
- Create `JsonPayload` and `XmlPayload` extending `DataPayload`.

2. The Generic Processor (Upper Bounds):

- Create a generic class `PipelineProcessor<T>`.
- *Constraint:* You must use an **Upper Bounded Type Parameter** (`<T DataPayload extends>`) so the generic class rejects `Strings`, `Integers`, or anything that isn't a valid payload.
- Implement a method `public void process(T payload)`. Because of the upper bound constraint, you must be able to call `.getRawContent()` directly on the generic `T` object without casting it.

Why this makes a great interview/study question:

- **Compile-Time Safety over Runtime Crashes:** Without Generics, collections and processors accept `Object`, requiring manual downcasting (`JsonPayload`) obj. If the wrong object is passed, it compiles fine but crashes with a `ClassCastException` in production. Generics push these errors to compile-time.
- **Bounded Parameters:** It perfectly illustrates why `<T>` is sometimes not enough. If it was just `<T>`, the compiler wouldn't know that `T` has a `getRawContent()` method, and would throw an error inside the `process` method. The `extends` keyword acts as a guarantee to the compiler.

Problem 1: The Polymorphic State Illusion (Early vs. Late Binding)

Enterprise Scenario: An accounting microservice uses an upcasted parent reference to process premium accounts. A junior developer attempts to read a configuration property directly from the object field instead of using a getter method, resulting in incorrect interest rate calculations.

Task: Write a program featuring a `BaseAccount` class and a `PremiumAccount` subclass. Both classes must declare an `interestRate` variable and a `getInterestRate()` method. Upcast the `PremiumAccount` to `BaseAccount`. Print both the variable directly and the method return value, then explain the discrepancy using JVM binding rules.

Code Breakdown & Memory Analysis:

- `account.interestRate` outputs 4.5. **Variables do not support polymorphism.** The JVM resolves instance variables at *compile-time* (Early Binding) based entirely on the reference type in the Stack (`BaseAccount`).
 - `account.getInterestRate()` outputs 7.5. **Methods support polymorphism.** The JVM resolves overridden methods at *runtime* (Late Binding / Dynamic Method Dispatch). It looks past the Stack reference, checks the actual object in the Heap (`PremiumAccount`), looks up its vtable, and invokes the child's specific implementation.
-

Problem 2: The Premature Dispatch Trap (Constructor Chaining)

Enterprise Scenario: A configuration service relies on a parent class to set up a database connection. However, a child class overrides the setup method to inject a custom authentication token. The application crashes with a NullPointerException on startup.

Task: Create an abstract parent class DatabaseConfig that calls an overridden setup() method *inside its own constructor*. Create a child class SecureConfig that overrides setup() to print a local String authToken variable initialized to "Bearer123". Execute the child class and explain the JVM memory initialization order causing the null output.

Code Breakdown & Memory Analysis:

- When new SecureConfig() is executed, memory is allocated on the Heap, and all reference variables (like authToken) default to null.
 - Constructor chaining forces the JVM to execute DatabaseConfig() first via super().
 - Inside DatabaseConfig(), it triggers setup(). Because of Dynamic Method Dispatch, the JVM executes the *Child's* setup() method.
 - However, the SecureConfig constructor hasn't run yet! The assignment authToken = "Bearer123" has not occurred. The method prints null. This proves that while Heap allocation happens instantly, state initialization happens strictly top-down through the constructor chain.
-

Problem 3: The Default Method Diamond Collision

Enterprise Scenario: As a system migrated to Java 8, architects added default logging mechanisms to multiple interfaces to avoid breaking existing code. A new core service implements two different interfaces, both of which provide a default implementation for a `log()` method, causing the compilation to fail entirely.

Task: Write an `AuditTracker` interface and a `PerformanceTracker` interface, both featuring a default `void log()` method. Write a `MasterService` class that implements both interfaces. Resolve the resulting "Diamond Problem" compiler error using Java's specific `super` keyword syntax for interface resolution.

Code Breakdown & Memory Analysis:

- Java classes cannot use multiple inheritance to avoid the Diamond Problem (ambiguity over which implementation to execute). When Java 8 introduced default methods in interfaces, this problem re-emerged.
 - The JVM detects the collision at compile-time. To fix it, the implementing class *must* override the conflicting method, providing its own implementation.
 - To reuse the logic from the interfaces without duplicating code, we use `InterfaceName.super.methodName()`. This allows precise, unambiguous Late Binding routing in memory.
-

Problem 4: Dynamic Strategy Orchestrator (Polymorphic Arrays)

Enterprise Scenario: An e-commerce checkout engine must apply multiple, dynamic pricing strategies to a shopping cart. The strategies applied change depending on the user's VIP status and the current season.

Task: Create a PricingStrategy interface with a calculate(double total) method. Implement BlackFridayStrategy (20% off) and VIPStrategy (\$50 flat off). In a main method, load these strategies into a polymorphic array. Loop through the array, applying each strategy sequentially to a base total of \$500, demonstrating loose coupling.

Code Breakdown & Memory Analysis:

- PricingStrategy[] activeDiscounts: This array only stores memory addresses (references) in the Heap. The JVM doesn't care what the actual objects are, so long as their class implements PricingStrategy.
 - Inside the loop, strategy.calculate() relies on Dynamic Method Dispatch. The Checkout code is entirely closed to modification but open to extension (Open/Closed Principle). You can add 50 new discount classes tomorrow without changing the for loop logic.
-

Problem 5: The Downcasting Minefield (instanceof verification)

Enterprise Scenario: A batch processing worker receives a polymorphic `List<Vehicle>` containing various subclasses. Only electric vehicles need a specific firmware update. If a developer blindly downcasts all vehicles to `ElectricCar` to call `updateFirmware()`, the application will crash violently in production.

Task: Create an abstract `Vehicle` class, a `GasCar` subclass, and an `ElectricCar` subclass featuring a unique `updateFirmware()` method. Create an array of generic `Vehicle` references holding both types. Safely iterate the array, checking object types before executing the downcast to avoid a `ClassCastException`.

Code Breakdown & Memory Analysis:

- `v.drive()` works because the compiler verifies `drive()` exists in the `Vehicle` reference class.
- `v.updateFirmware()` causes a Compile-Time Error because the Stack reference (`Vehicle`) does not possess this method blueprint.
- Blindly forcing it (`((ElectricCar) v)`) overrides the compiler, but when the JVM examines the Heap at runtime and finds a `GasCar` object, it throws a `ClassCastException`.
- `instanceof` acts as a memory inspector. It asks the JVM to look at the physical object residing in the Heap. If the heap payload structurally matches the `ElectricCar` class blueprint, it grants permission to safely repoint an `ElectricCar` Stack reference to that specific Heap object.